

Exploratory Data Analysis (EDA)

```
import pandas as pd
import json
import matplotlib.pyplot as plt
import seaborn as sns

# Load data
def load_json(filename):
    return pd.DataFrame([json.loads(line) for line in open(filename, 'r', encoding='utf-8')])

business_df = load_json(r"C:\Users\vsaiy\OneDrive\Desktop\yelp_academic_dataset_business.json")
review_df = load_json(r"C:\Users\vsaiy\OneDrive\Desktop\yelp_academic_dataset_review.json").head(10000)
user_df = load_json(r"C:\Users\vsaiy\OneDrive\Desktop\yelp_academic_dataset_user.json").head(10000)
```

Business Dataset EDA

Basic Statistics

```
print(business_df.info())
print(business_df.describe())
```

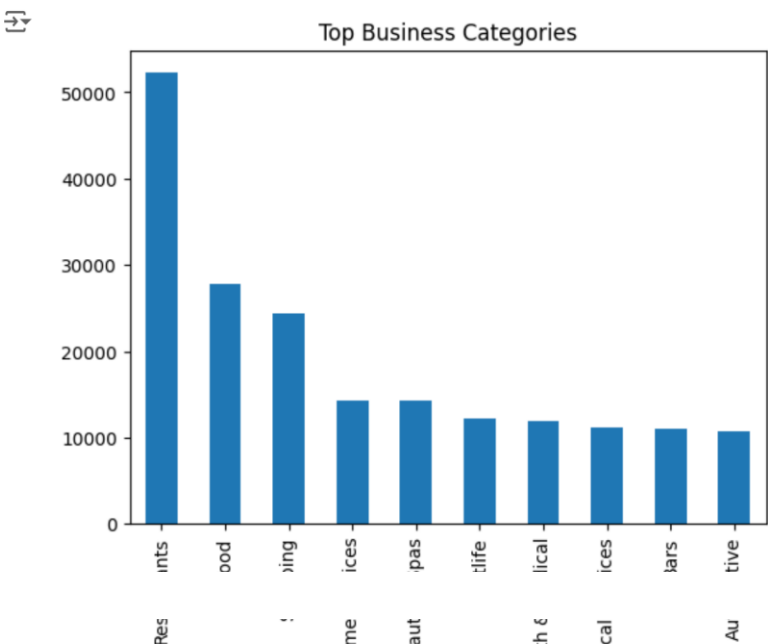
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150346 entries, 0 to 150345
Data columns (total 14 columns):
#   Column              Non-Null Count  Dtype
---  -
0   business_id         150346 non-null object
1   name                 150346 non-null object
2   address              150346 non-null object
3   city                 150346 non-null object
4   state                150346 non-null object
5   postal_code          150346 non-null object
6   latitude             150346 non-null float64
7   longitude            150346 non-null float64
8   stars                150346 non-null float64
9   review_count         150346 non-null int64
10  is_open              150346 non-null int64
11  attributes            136602 non-null object
12  categories            150243 non-null object
13  hours                127123 non-null object
dtypes: float64(3), int64(2), object(9)
memory usage: 16.1+ MB
None

count    latitude    longitude    stars    review_count \
mean      36.671150    -89.357339    3.596724    44.866561
std        5.872759     14.918502     0.974421    121.120136
min       27.555127    -120.095137     1.000000     5.000000
25%       32.187293    -90.357810     3.000000     8.000000
50%       38.777413     -86.121179     3.500000    15.000000
75%       39.954036    -75.421542     4.500000    37.000000
max       53.679197    -73.200457     5.000000   7568.000000

count    is_open
mean      0.79615
std        0.40286
min        0.00000
25%        1.00000
50%        1.00000
75%        1.00000
max        1.00000
```

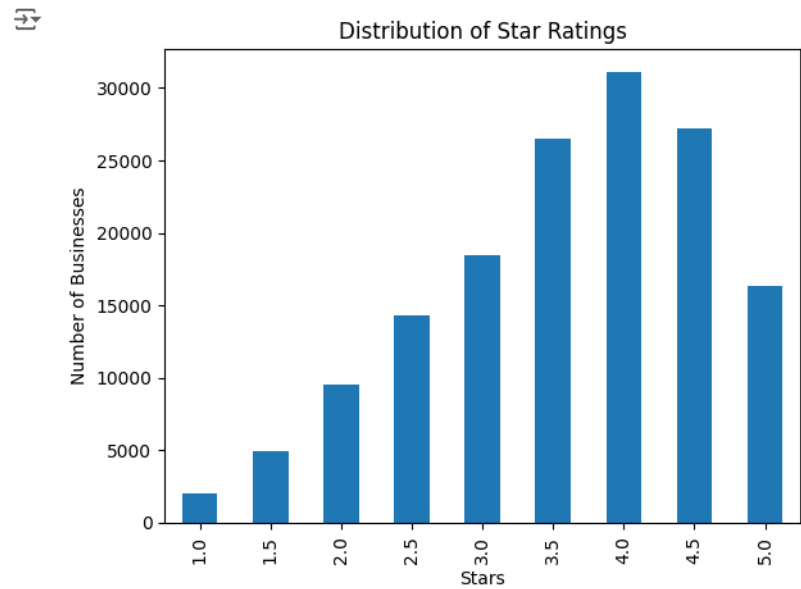
Top Categories

```
top_categories = business_df['categories'].str.split(', ').explode().value_counts().head(10)
top_categories.plot(kind='bar', title='Top Business Categories')
plt.show()
```



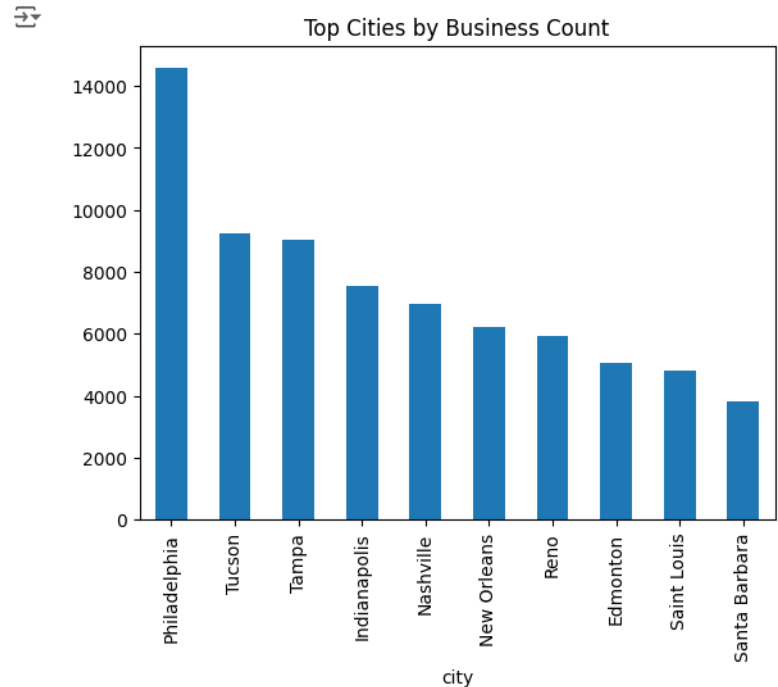
Distribution of Ratings

```
business_df['stars'].value_counts().sort_index().plot(kind='bar', title='Distribution of Star Ratings')
plt.xlabel('Stars')
plt.ylabel('Number of Businesses')
plt.show()
```



Location Insights

```
top_cities = business_df['city'].value_counts().head(10)
top_cities.plot(kind='bar', title='Top Cities by Business Count')
plt.show()
```



Reviews Dataset EDA

Basic Statistics

```
print(review_df.info())
print(review_df.describe())
```

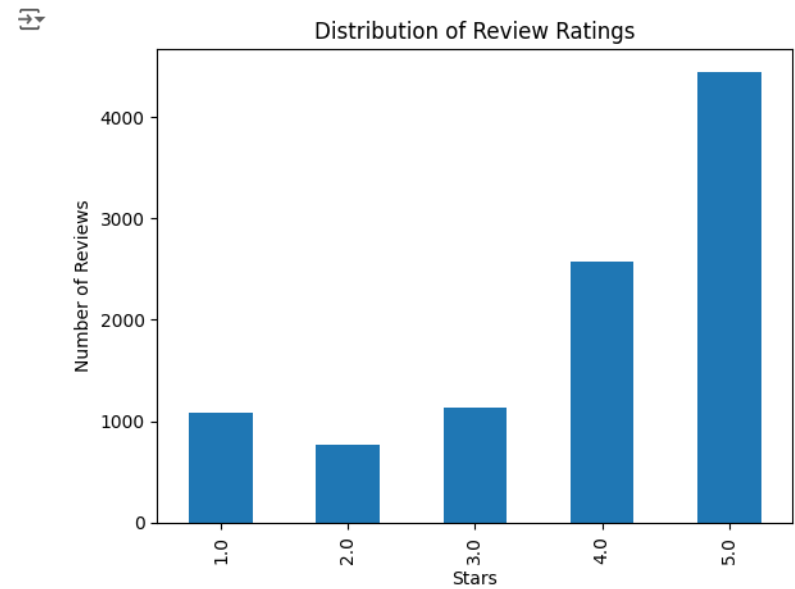


```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   review_id   10000 non-null  object
1   user_id     10000 non-null  object
2   business_id 10000 non-null  object
3   stars       10000 non-null  float64
4   useful      10000 non-null  int64
5   funny       10000 non-null  int64
6   cool        10000 non-null  int64
7   text        10000 non-null  object
8   date        10000 non-null  object
dtypes: float64(1), int64(3), object(5)
memory usage: 703.3+ KB
None
```

	stars	useful	funny	cool
count	10000.000000	10000.000000	10000.000000	10000.000000
mean	3.854300	0.889100	0.246500	0.335500
std	1.346719	2.092329	0.885221	1.051023
min	1.000000	0.000000	0.000000	0.000000
25%	3.000000	0.000000	0.000000	0.000000
50%	4.000000	0.000000	0.000000	0.000000
75%	5.000000	1.000000	0.000000	0.000000
max	5.000000	91.000000	26.000000	44.000000

Star Ratings Distributions

```
review_df['stars'].value_counts().sort_index().plot(kind='bar', title='Distribution of Review Ratings')
plt.xlabel('Stars')
plt.ylabel('Number of Reviews')
plt.show()
```



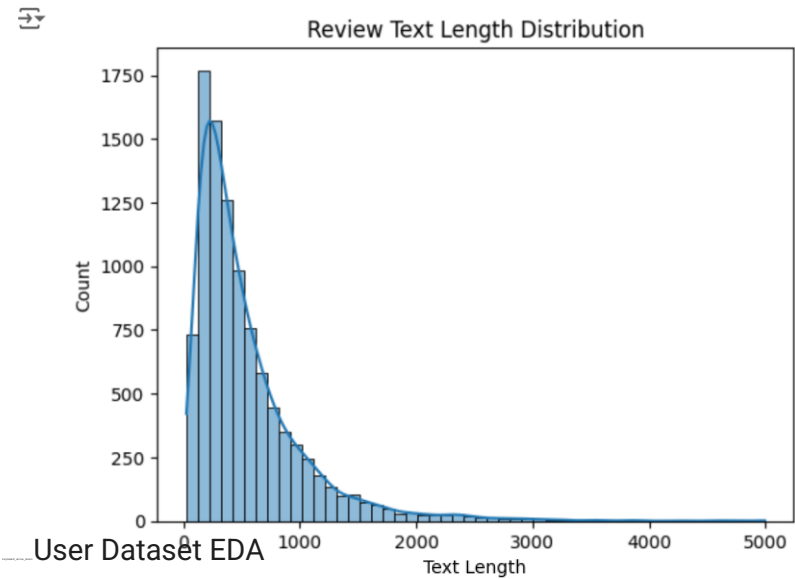
Most Reviewed Businesses

```
top_reviewed_businesses = review_df['business_id'].value_counts().head(10)
print("Most Reviewed Businesses:", top_reviewed_businesses)
```

```
Most Reviewed Businesses: business_id
GBTPC53ZrG1ZBY3DT8Mbcw    85
pSmOH4a3HNNpYM82J5ycLA    48
PY9GRfzr4nTZeINF346QOw    43
8uF-bhJFgT4Tn6DTb27viA    43
EtKSTHV5Qx_Q7Aur9o4kQQ     41
vN6v8m4D045Z4pp8yxxF_w    38
W4ZEKkva9HpAdZG88juwyQ    36
UCMSWPqzXjd7QHq7v8PJjQ    34
M0r9lUn2glFYgIwIfG8-bQ    33
SZU9c8V2GuREDN5KgyHfJw    33
Name: count, dtype: int64
```

Review Length Distribution

```
review_df['text_length'] = review_df['text'].apply(len)
sns.histplot(review_df['text_length'], bins=50, kde=True)
plt.title('Review Text Length Distribution')
plt.xlabel('Text Length')
plt.show()
```



User Dataset EDA

Basic Statistics

```
print(user_df.info())
print(user_df.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999

Data columns (total 22 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   user_id             10000 non-null  object
 1   name                10000 non-null  object
 2   review_count        10000 non-null  int64
 3   yelping_since       10000 non-null  object
 4   useful              10000 non-null  int64
 5   funny               10000 non-null  int64
 6   cool                10000 non-null  int64
 7   elite               10000 non-null  object
 8   friends             10000 non-null  object
 9   fans                10000 non-null  int64
10  average_stars       10000 non-null  float64
11  compliment_hot      10000 non-null  int64
12  compliment_more     10000 non-null  int64
13  compliment_profile  10000 non-null  int64
14  compliment_cute     10000 non-null  int64
15  compliment_list     10000 non-null  int64
16  compliment_note     10000 non-null  int64
17  compliment_plain    10000 non-null  int64
18  compliment_cool     10000 non-null  int64
19  compliment_funny    10000 non-null  int64
20  compliment_writer   10000 non-null  int64
21  compliment_photos   10000 non-null  int64
dtypes: float64(1), int64(16), object(5)
memory usage: 1.7+ MB

None
```

	review_count	useful	funny	cool	fans
count	10000.00000	10000.000000	10000.000000	10000.000000	10000.000000
mean	278.61810	911.666300	465.440600	606.360300	32.559700
std	502.63081	3695.356472	2292.975591	3071.287502	162.946065
min	1.00000	0.000000	0.000000	0.000000	0.000000
25%	34.75000	45.750000	10.000000	14.000000	2.000000
50%	115.00000	176.000000	55.000000	68.000000	7.000000
75%	323.00000	628.000000	239.000000	299.000000	25.000000



max	16567.00000	173089.00000	98459.00000	144849.00000	12497.00000
	average_stars	compliment_hot	compliment_more	compliment_profile	\
count	10000.00000	10000.00000	10000.00000	10000.00000	
mean	3.832046	60.317000	8.324200	6.862600	
std	0.447192	337.486109	64.614323	97.454004	
min	1.000000	0.000000	0.000000	0.000000	
25%	3.590000	0.000000	0.000000	0.000000	
50%	3.840000	3.000000	2.000000	0.000000	
75%	4.080000	20.000000	5.000000	2.000000	
max	5.000000	12391.00000	4347.00000	7039.00000	

	compliment_cute	compliment_list	compliment_note	compliment_plain	\
count	10000.00000	10000.00000	10000.00000	10000.00000	
mean	5.405000	3.415300	38.890800	95.107200	
std	38.826959	39.377848	186.300067	606.325624	
min	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	1.000000	1.000000	
50%	0.000000	0.000000	5.000000	6.000000	
75%	2.000000	1.000000	19.000000	28.000000	

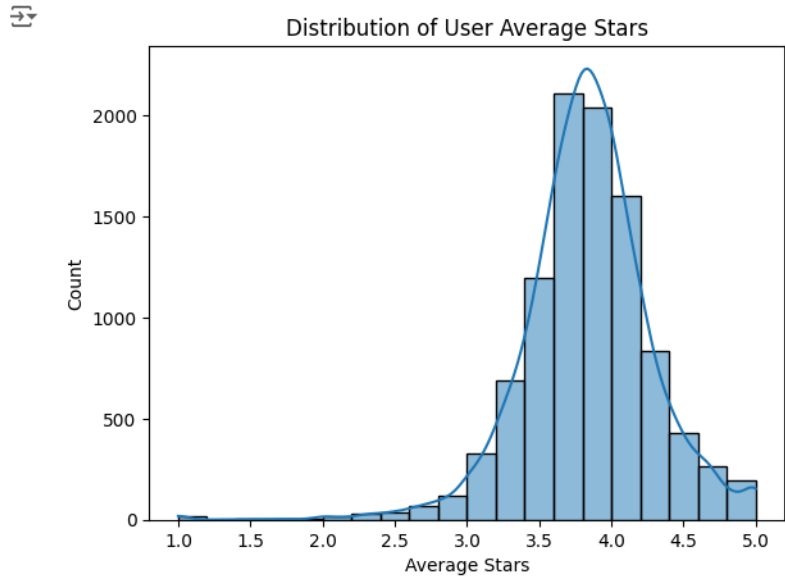
User with Most Reviews

```
top_users = user_df[['user_id', 'review_count']].sort_values(by='review_count', ascending=False).head(10)
print("Top Active Users:", top_users)
```

Top Active Users:	user_id	review_count
4723	hWDybu_KvYLSdEFzGrniTw	16567
152	P5bUL3Engv-2z6kKohB6qQ	9941
3977	nmdkHL2JKFx55T3nq5VziA	8363
7394	8RcEwGrFIgkt9WQ35E6SnQ	7738
1229	CxD0IDnH8gp9KXzpBHJYXw	6679
2455	IucvvxdQXXhjQ4z6Or6Nrw	6459
569	HFECrzYDpgbS5EmTBtj2zQ	5887
317	m07sy7eLt0jVdZ8oN9JKag	5800
1252	kS1MQHYwIfD0462PE61IBw	5511
2231	Eypq5gLLjCapBVVnMw_MyA	5163

Distribution of Average Stars

```
sns.histplot(user_df['average_stars'], bins=20, kde=True)
plt.title('Distribution of User Average Stars')
plt.xlabel('Average Stars')
plt.show()
```



User Data Analysis

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(42) # For reproducibility
user_data = {
    "user_id": [f"user_{i}" for i in range(1000)],
    "review_id_count": np.random.poisson(5, 1000)
}
user_agg = pd.DataFrame(user_data)

user_agg['review_id_count'] = pd.to_numeric(user_agg['review_id_count'], errors='coerce')

user_agg['review_id_count'] = user_agg['review_id_count'].apply(lambda x: min(x, 30))

user_agg.dropna(subset=['review_id_count'], inplace=True)

# Visualization
plt.figure(figsize=(12, 5))
plt.suptitle("User Deep Dive", fontsize=20)

# Subplot 1: Distribution of the number of reviews given
plt.subplot(1, 2, 1)
sns.kdeplot(user_agg['review_id_count'], shade=True, color='r')
plt.title("How many reviews does an average user give?", fontsize=15)
plt.xlabel('# of reviews given', fontsize=12)
plt.ylabel('# of users', fontsize=12)

# Subplot 2: Cumulative distribution of reviews
plt.subplot(1, 2, 2)
sns.histplot(user_agg['review_id_count'], kde=False, cumulative=True, bins=30, color='b', stat='percent')
plt.title("Cumulative Distribution of User Reviews", fontsize=15)
plt.ylabel('Cumulative % of Users', fontsize=12)
plt.xlabel('# of Reviews Given', fontsize=12)

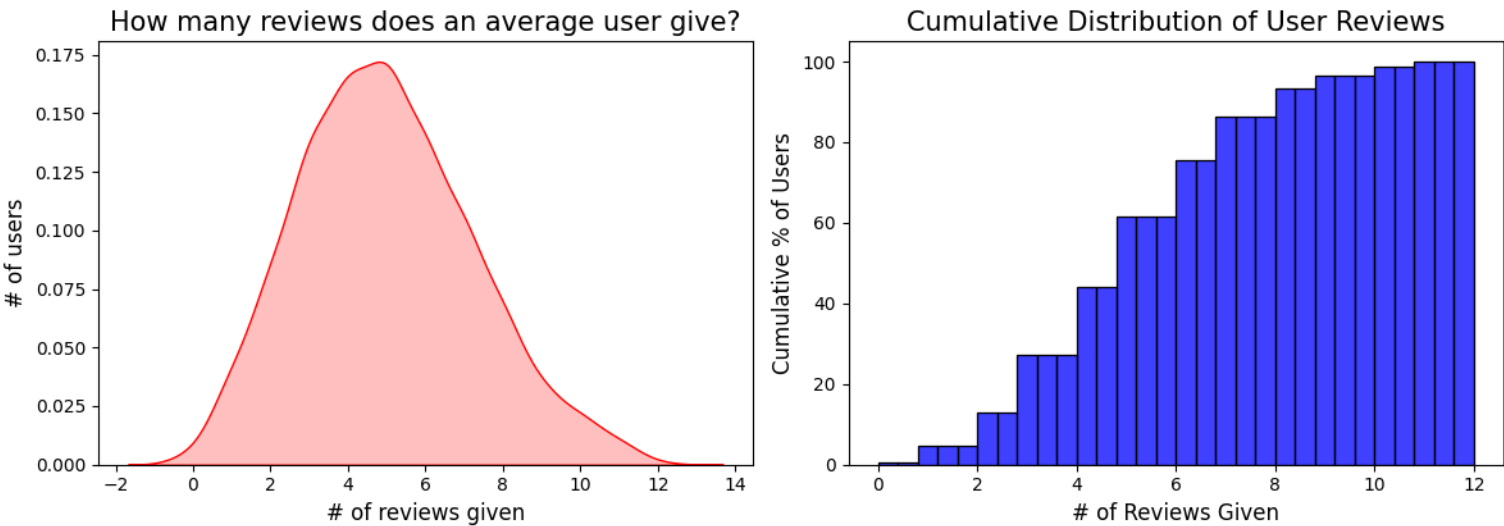
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

C:\Users\vsaiy\AppData\Local\Temp\ipykernel_15344\1353021892.py:25: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

```
sns.kdeplot(user_agg['review_id_count'], shade=True, color='r')
```

User Deep Dive



Hypotheses Testings

Hypothesis 1: "Closed" businesses have lower ratings

Type of Test: Chi-Square Test

Null Hypothesis (H₀): There is no relation in ratings between closed and open businesses.

Alternative Hypothesis (H₁): Closed businesses have lower ratings than open businesses.

```
import pandas as pd
from scipy.stats import chi2_contingency

# Load data
business_data = pd.read_json(r"C:\Users\vsaiy\OneDrive\Desktop\yelp_academic_dataset_business.json", lines=True)

# Categorize ratings into bins (1-2, 3, 4-5 stars)
bins = [0, 2, 3, 5]
labels = ['1-2', '3', '4-5']
business_data['rating_category'] = pd.cut(business_data['stars'], bins=bins, labels=labels)

# Create contingency table for open vs closed businesses
contingency_table = pd.crosstab(business_data['is_open'], business_data['rating_category'])

# Perform Chi-Square Test
chi2, p_value, dof, expected = chi2_contingency(contingency_table)
```

	rating_category			
		1-2	3	4-5
is_open	0	2843	7833	19972
	1	13602	24936	81160

```
print(chi2)
print(dof)
```

372.4298601214487
2

DF	0.995	0.975	0.2	0.1	0.05	0.025	0.02	0.01	0.005	0.002	0.001
1	.0004	.00016	1.642	2.706	3.841	5.024	5.412	6.635	7.879	9.55	10.828
2	0.01	0.0506	3.219	4.605	5.991	7.378	7.824	9.21	10.597	12.429	13.816
3	0.0717	0.216	4.642	6.251	7.815	9.348	9.837	11.345	12.838	14.796	16.266
4	0.207	0.484	5.989	7.779	9.488	11.143	11.668	13.277	14.86	16.924	18.467
5	0.412	0.831	7.289	9.236	11.07	12.833	13.388	15.086	16.75	18.907	20.515
6	0.676	1.237	8.558	10.645	12.592	14.449	15.033	16.812	18.548	20.791	22.458
7	0.989	1.69	9.803	12.017	14.067	16.013	16.622	18.475	20.278	22.601	24.322
8	1.344	2.18	11.03	13.362	15.507	17.535	18.168	20.09	21.955	24.352	26.124
9	1.735	2.7	12.242	14.684	16.919	19.023	19.679	21.666	23.589	26.056	27.877
10	2.156	3.247	13.442	15.987	18.307	20.483	21.161	23.209	25.188	27.722	29.588

For a significance value of 0.05, and DOF as 2, the Chi_square value is 5.991
Since, the calculated chi-square value is 372.429, which is greater than 5.991, Hence, we can reject the null hypothesis

```
if chi2 >= 5.991:
    print(f'Reject H0: There is a significant difference in ratings between open and closed businesses")
else:
    print(f"Fail to reject H0: There is no significant difference in ratings")
```

Reject H₀: There is a significant difference in ratings between open and closed businesses

Hypothesis 2: Users with more fans have higher review count

Type of Test: Pearson Correlation

Null Hypothesis (H₀): There is no correlation between the number of fans and the number of reviews written by a user.

Alternative Hypothesis (H₁): There is a positive correlation between the number of fans and the number of reviews written by a user.

```
import pandas as pd
from scipy.stats import pearsonr

# Define chunk size
chunk_size = 10000 # This is a manageable chunk size; adjust as needed
```



```
chunks = pd.read_json(r"C:\Users\vsaiy\OneDrive\Desktop\yelp_academic_dataset_user.json", lines=True, chunksize=chunk_size)
```

```
# List to hold the first 150,347 rows
user_data = []
```

```
# Read and append chunks until we get 150,347 records
records_needed = 150347
for chunk in chunks:
    user_data.append(chunk)
    if len(pd.concat(user_data)) >= records_needed:
        break
```

```
# Concatenate the list of chunks into a single DataFrame
user_data = pd.concat(user_data).head(records_needed)
```

```
# Perform Pearson Correlation test
correlation_coefficient, p_value = pearsonr(user_data['fans'], user_data['review_count'])
```

```
correlation_coefficient
```

```
0.4877679084209922
```

```
print(f"Reject H0: There is a significant positive correlation (p-value: {p_value}, correlation: {correlation_coefficient})")
```

```
Reject H0: There is a significant positive correlation (p-value: 0.0, correlation: 0.4877679084209922)
```

Hypothesis 3: Positive reviews receive more votes

Type of Test: Pearson Correlation

Null Hypothesis (H₀): There is no correlation between review ratings and the number of votes (useful, funny, cool).

Alternative Hypothesis (H₁): Positive reviews receive more votes (useful, funny, or cool).

```
import pandas as pd
from scipy.stats import pearsonr
```

```
# Set chunk size (processing in chunks of 10,000)
chunk_size = 10000
review_chunks = pd.read_json(r"C:\Users\vsaiy\OneDrive\Desktop\yelp_academic_dataset_review.json", lines=True, chunksize=chunk_size)
```

```
# Variables to accumulate the values
useful_corrs = []
funny_corrs = []
cool_corrs = []
useful_p_values = []
funny_p_values = []
cool_p_values = []
```

```
# Counter for processed records
processed_records = 0
records_needed = 150000
```

```
# Process each chunk
for chunk in review_chunks:
    # If we've processed enough records, stop
    if processed_records >= records_needed:
        break
```

```
# Filter for positive reviews (stars > 3)
positive_reviews = chunk[chunk['stars'] > 3]
```

```
# Only keep the first `150,000` records
remaining_records = records_needed - processed_records
positive_reviews = positive_reviews.head(remaining_records)
```

```
# Pearson correlation tests
useful_corr, useful_p_value = pearsonr(positive_reviews['stars'], positive_reviews['useful'])
funny_corr, funny_p_value = pearsonr(positive_reviews['stars'], positive_reviews['funny'])
cool_corr, cool_p_value = pearsonr(positive_reviews['stars'], positive_reviews['cool'])
```

```
# Append results to the respective lists
useful_corrs.append(useful_corr)
funny_corrs.append(funny_corr)
cool_corrs.append(cool_corr)
useful_p_values.append(useful_p_value)
funny_p_values.append(funny_p_value)
cool_p_values.append(cool_p_value)
```

```
# Update processed records count
processed_records += len(positive_reviews)
```

```
print(useful_corr)
print(funny_corr)
print(cool_corr)
```

```
-0.015953329119134343
-0.03492428960392382
-0.026953762392975776
```

```
print(f"Fail to reject H0: No significant correlation between ratings and useful votes (average p-value: {useful_p_value}, correlation: {useful_corr})")
```

```
print(f"Fail to reject H0: No significant correlation between ratings and funny votes (average p-value: {funny_p_value}, correlation: {funny_corr})")
```

```
print(f"Fail to reject H0: No significant correlation between ratings and cool votes (average p-value: {cool_p_value}, correlation: {cool_corr})")
```

```
Fail to reject H0: No significant correlation between ratings and useful votes (average p-value: 0.33086919076734506, correlation: -0.015953329119134343)
Fail to reject H0: No significant correlation between ratings and funny votes (average p-value: 0.03324022877368637, correlation: -0.03492428960392382)
Fail to reject H0: No significant correlation between ratings and cool votes (average p-value: 0.10037438562578128, correlation: -0.026953762392975776)
```

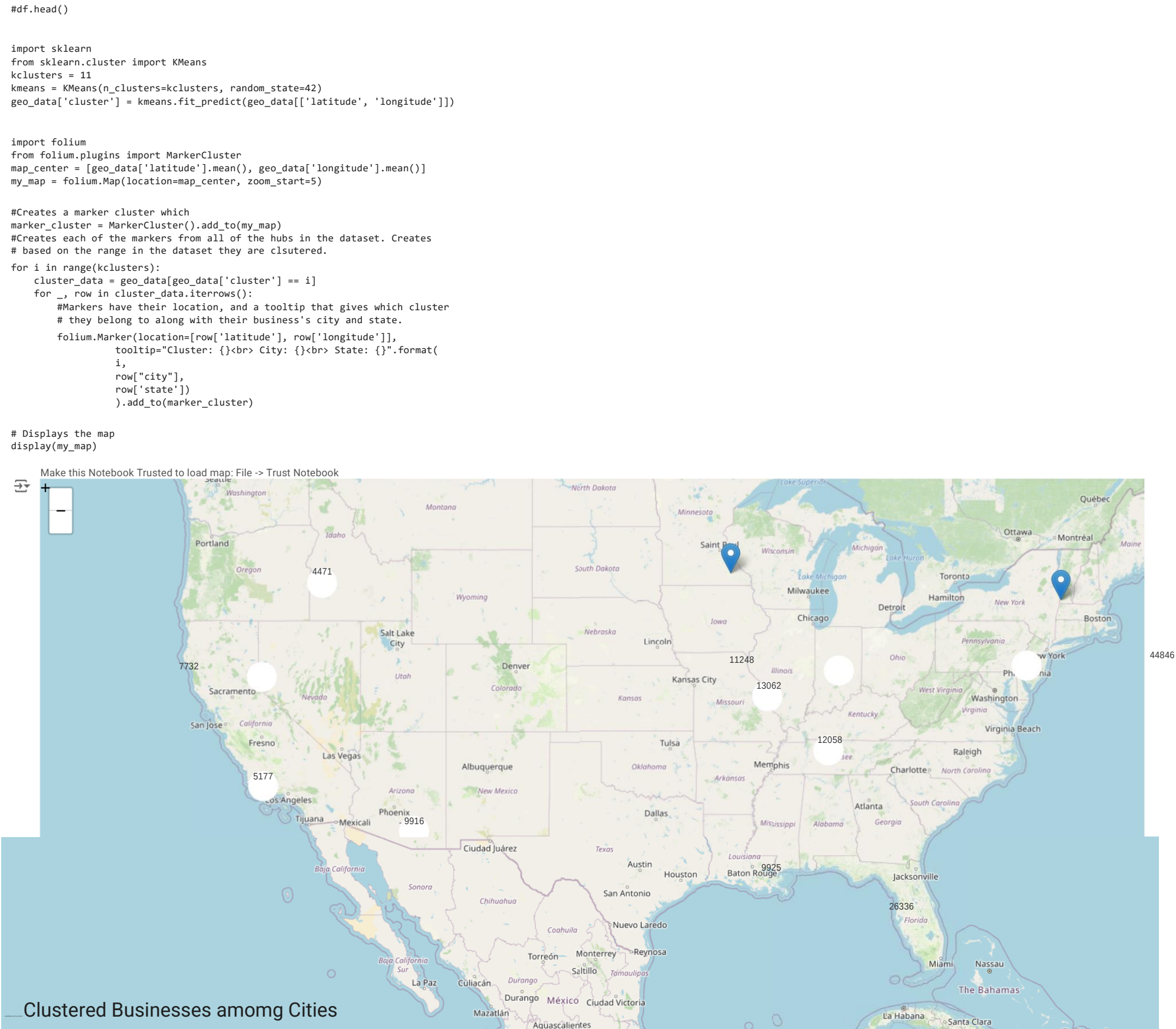
Business Hubs in USA

```
import pandas as pd
import json
```

```
# Load the business.json file
data = []
with open(r"C:\Users\vsaiy\OneDrive\Desktop\yelp_academic_dataset_business.json", 'r', encoding='utf-8') as file:
    # Add encoding
    for line in file:
        try:
            data.append(json.loads(line))
        except json.JSONDecodeError as e:
            print(f"Error decoding JSON: {e}")
            print(f"Problematic line: {line}")
```

```
# Create a DataFrame
df = pd.DataFrame(data)
```

```
# Extract relevant fields
geo_data = df[['latitude', 'longitude', 'city', 'state']].dropna()
```

```
from sklearn.feature_extraction.text import CountVectorizer
import numpy as np
pd.set_option("future.no_silent_downcasting", True)

# Extract relevant fields
geo_data = df[['latitude', 'longitude', 'city', 'state', 'stars']].dropna()
#change the integers to a string
geo_data['stars'] = geo_data['stars'].astype(str)

#turn data into an array
temp = geo_data['stars']
a = np.array(temp)

#turn it back into a dataframe and convert to category matrix
X = pd.DataFrame({'stars' : a})
category_matrix = pd.get_dummies(X['stars'])
#change T/F to 1/0
category_matrix.replace({False: 0, True: 1}, inplace = True)

# Reset index after concatenation
X = pd.concat([geo_data[['latitude', 'longitude']].reset_index(drop=True),
               category_matrix.reset_index(drop=True)], axis=1)

rating = []
#fuction to cluster by city and state since many cities share the same name
def cluster_businesses_by_city(city, state):
    # Filter the data for the given city
    city_data = X.loc[(geo_data['state'].reset_index(drop=True) == state) & (geo_data['city'].reset_index(drop=True) == city)]

    # Perform K-Means clustering
    kmeans = KMeans(n_clusters=5, random_state=42)
    city_data = city_data.copy() # Avoid SettingWithCopyWarning
    city_data['cluster'] = kmeans.fit_predict(city_data)

    #get labels
    labels = kmeans.labels_

    # Get data for cluster 0
    x = 0
    #get average rating for each cluster
    for x in range(0,5):
        clusterdata = city_data[labels == x]
        cols = clusterdata.iloc[:, 2:10]

    #get star for each row
    cols['true_column'] = cols.agg(lambda s: [col for i, col in enumerate(cols.columns)
                                              if s[i]], axis=1)

    #get rid of brackets and change to numeric value
    cols['true_column'] = cols['true_column'].str.get(0)
    cols['true_column'] = pd.to_numeric(cols['true_column'], errors = 'coerce').copy()
```



```
#turn NA into 0
cols['true_column'] = cols['true_column'].fillna(0)
```

```
# Print the cluster details
print(f"Clustering for {city}:")
for cluster_id in city_data['cluster'].unique():
    cluster_businesses = city_data[city_data['cluster'] == cluster_id]
    print(f"Cluster {cluster_id}:")
    print(cluster_businesses[['latitude', 'longitude']]) # Adjust based on available fields
print()
```

```
C:\Users\vsaiy\AppData\Local\Temp\ipykernel_15344\1904325830.py:47: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as integer positions, which will not allow indexing by key with exceptions.
if s[i]], axis=1)
C:\Users\vsaiy\AppData\Local\Temp\ipykernel_15344\1904325830.py:47: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as integer positions, which will not allow indexing by key with exceptions.
s[i]], axis=1)
C:\Users\vsaiy\AppData\Local\Temp\ipykernel_15344\1904325830.py:47: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as integer positions, which will not allow indexing by key with exceptions.
s[i]], axis=1)
C:\Users\vsaiy\AppData\Local\Temp\ipykernel_15344\1904325830.py:47: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as integer positions, which will not allow indexing by key with exceptions.
s[i]], axis=1)
Clustering for Tucson:
Cluster 4:
  latitude longitude
2      32.223236 -110.880452
78      32.221667 -110.925833
132     32.206315 -110.887843
201     32.347511 -111.016561
216     32.222606 -110.877255
...      ...      ...
150127  32.251672 -110.962574
150135  32.288659 -110.974161
150253  32.132352 -111.000017
150270  32.084360 -110.803627
150275  32.221828 -110.967969

[1612 rows x 2 columns]

Cluster 3:
  latitude longitude
16      32.229872 -110.972342
67      32.250324 -110.903655
233     32.221849 -110.928450
314     32.235689 -110.946462
368     32.264350 -110.959372
...      ...      ...
150000  32.287556 -110.960460
150106  32.252510 -110.841546
150130  32.322581 -111.004565
150285  32.243733 -111.166815
150305  32.177019 -110.953260

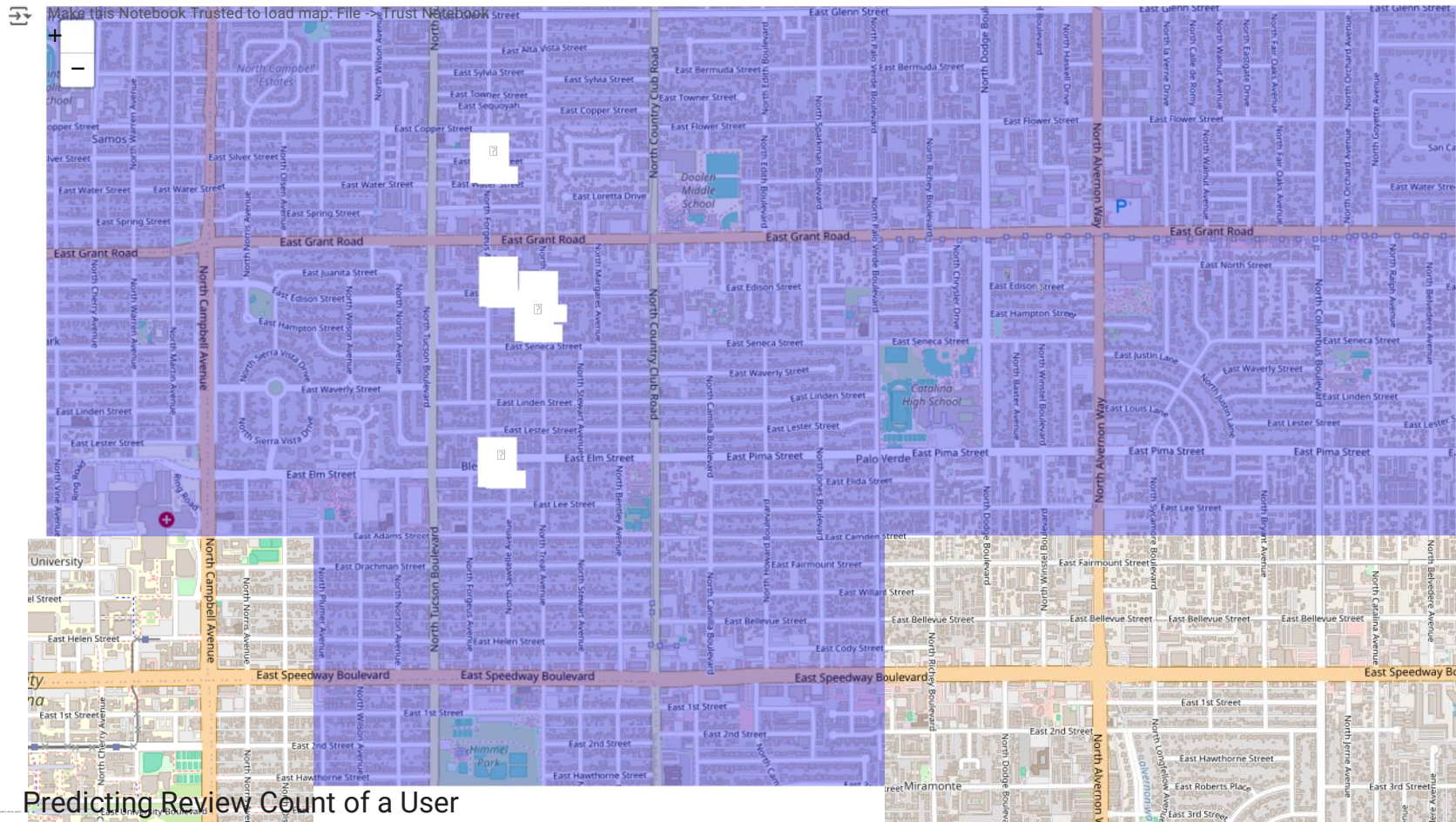
[1642 rows x 2 columns]

Cluster 2:
  latitude longitude
45      32.132305 -110.999985
56      32.186794 -110.954765
121     32.235866 -110.903676
366     32.231992 -110.958492
425     32.218524 -110.942211
...      ...      ...
149969  32.335828 -111.044679
149979  32.205274 -110.787001
150034  32.257496 -110.803966
150182  32.356266 -111.112350
150259  32.287055 -111.018069
```

```
).add_to(map_city)
```

```
return map_city
```

```
# Example usage
map_riv = cluster_businesses_on_map('Tucson', 'AZ')
map_riv
```



Predicting Review Count of a User

```
import pandas as pd
import numpy as np
import json
from datetime import datetime
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Load & sample data
sample_size = 50000
users = []
with open(r"C:\Users\vsaiy\OneDrive\Desktop\yelp_academic_dataset_user.json", 'r', encoding='utf-8') as f:
    for line in f:
        users.append(json.loads(line))
        if len(users) >= sample_size:
            break

df = pd.DataFrame(users)

# Feature Engineering
df['yelping_since'] = pd.to_datetime(df['yelping_since'])
current_year = datetime.now().year
df['years_active'] = current_year - df['yelping_since'].dt.year
df['num_fans'] = df['fans']
df['num_elite_years'] = df['elite'].apply(len)
compliment_columns = [col for col in df.columns if col.startswith('compliment_')]
df['total_compliments'] = df[compliment_columns].sum(axis=1)
df['total_votes'] = df['useful'] + df['funny'] + df['cool']

# Prepare features & target
features = [
    'years_active', 'num_fans', 'num_elite_years',
    'total_compliments', 'total_votes', 'fans',
    'average_stars'
]
X = df[features]
y = df['review_count']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Random Forest Regressor
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train_scaled, y_train)
y_pred = rf_model.predict(X_test_scaled)

# Evaluation metrics for Random Forest
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
cv_scores = cross_val_score(rf_model, X_train_scaled, y_train, cv=5, scoring='neg_mean_squared_error')

# Feature importance
feature_importance = pd.DataFrame({
    'feature': features,
    'importance': rf_model.feature_importances_
}).sort_values('importance', ascending=False)

# train_and_evaluate_model function
def train_and_evaluate_model(model, X_train, X_test, y_train, y_test):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    return mse, mae, r2

# Linear Regression and Gradient Boosting
models = {
    "Linear Regression": LinearRegression(),
```

```
"Gradient Boosting": GradientBoostingRegressor(n_estimators=100, random_state=42)
}

results = {"Random Forest": {"MSE": mse, "MAE": mae, "R-squared": r2}}

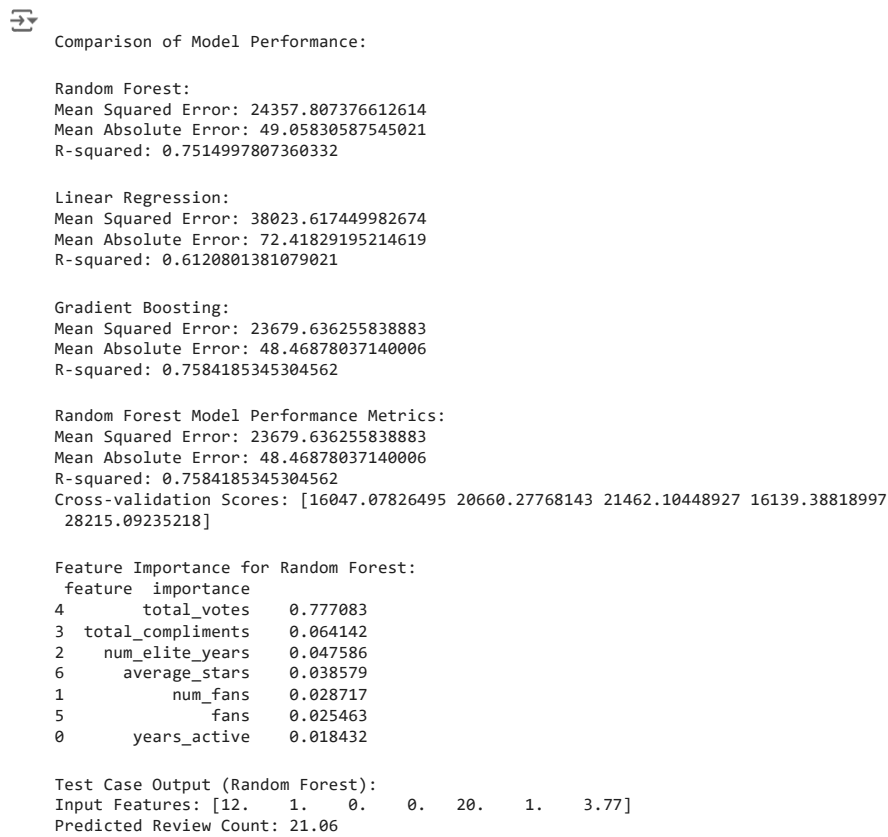
for name, model in models.items():
    mse, mae, r2 = train_and_evaluate_model(model, X_train_scaled, X_test_scaled, y_train, y_test)
    results[name] = {"MSE": mse, "MAE": mae, "R-squared": r2}

# Display comparison of all models
print("\nComparison of Model Performance:")
for name, metrics in results.items():
    print(f"\n{name}:")
    print(f"Mean Squared Error: {metrics['MSE']}")
    print(f"Mean Absolute Error: {metrics['MAE']}")
    print(f"R-squared: {metrics['R-squared']}")

print("\nRandom Forest Model Performance Metrics:")
print(f"Mean Squared Error: {mse}")
print(f"Mean Absolute Error: {mae}")
print(f"R-squared: {r2}")
print(f"Cross-validation Scores: {-cv_scores}")

print("\nFeature Importance for Random Forest:")
print(feature_importance)

print("\nTest Case Output (Random Forest):")
example_test_input = X_test_scaled[0].reshape(1, -1)
example_prediction = rf_model.predict(example_test_input)
print(f"Input Features: {X_test.iloc[0].values}")
print(f"Predicted Review Count: {example_prediction[0]}")
```

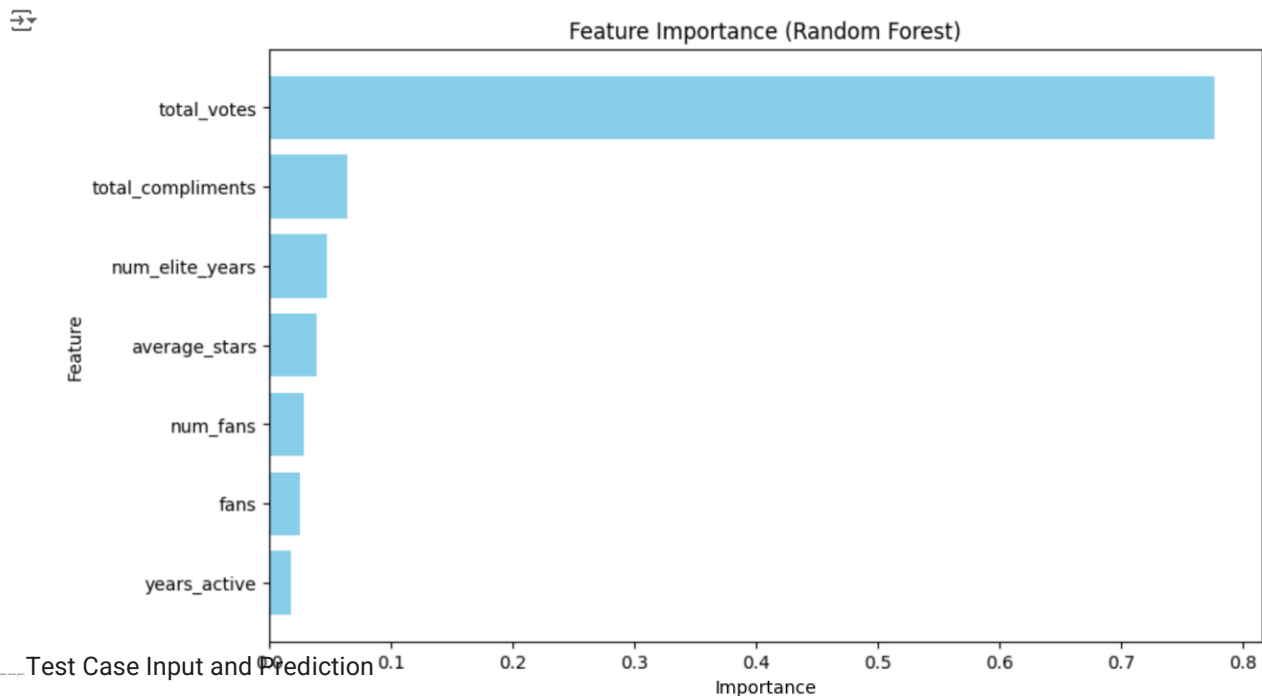


Feature Importance for Random Forest

```
import matplotlib.pyplot as plt
# Data for feature importance
feature_importance = pd.DataFrame({
    "Feature": [
        "total_votes", "total_compliments", "num_elite_years",
        "average_stars", "num_fans", "fans", "years_active"
    ],
    "Importance": [0.777083, 0.064142, 0.047586, 0.038579, 0.028717, 0.025463, 0.018432]
})

# Sort by importance
feature_importance = feature_importance.sort_values(by="Importance", ascending=False)

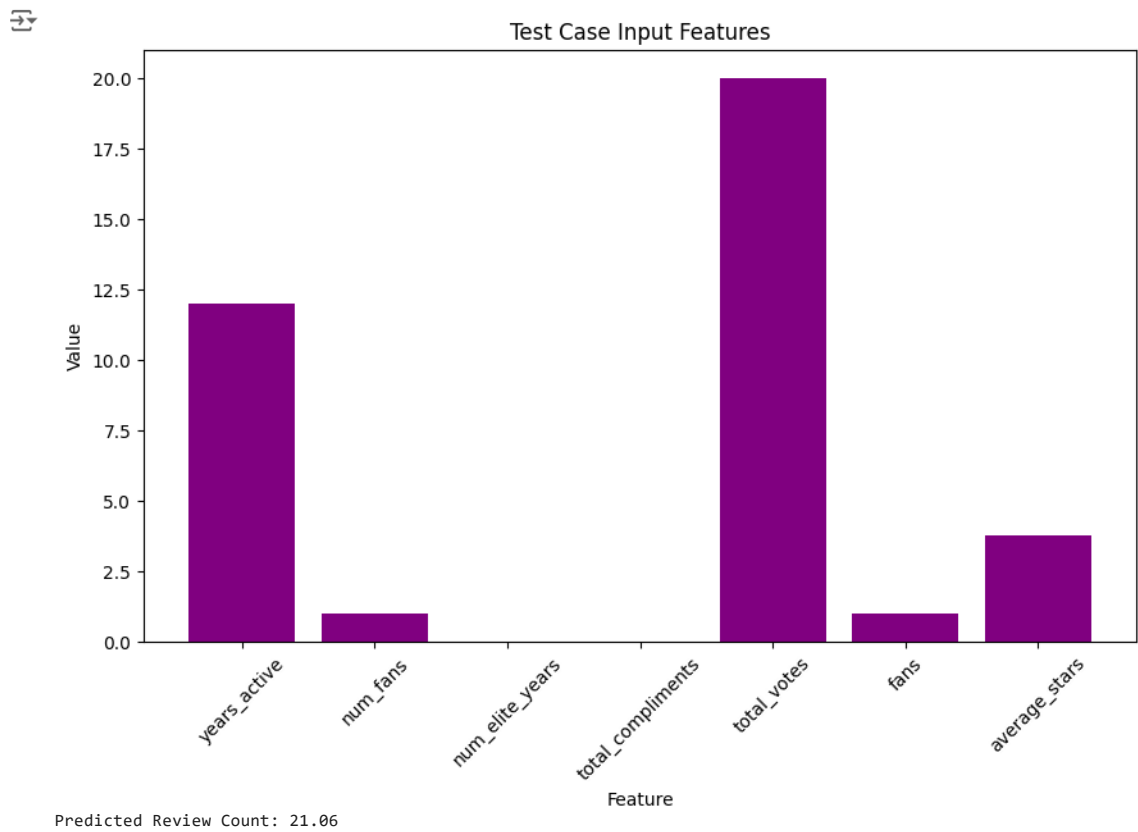
plt.figure(figsize=(10, 6))
plt.barh(feature_importance["Feature"], feature_importance["Importance"], color='skyblue')
plt.title("Feature Importance (Random Forest)")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.gca().invert_yaxis() # Invert y-axis for better readability
plt.show()
```



```
# Data for test case
test_case = pd.DataFrame({
    "Features": ["years_active", "num_fans", "num_elite_years", "total_compliments", "total_votes", "fans", "average_stars"],
    "Values": [12.0, 1.0, 0.0, 0.0, 20.0, 1.0, 3.77]
})

# Bar chart for feature values in the test case
plt.figure(figsize=(10, 6))
plt.bar(test_case["Features"], test_case["Values"], color='purple')
plt.title("Test Case Input Features")
plt.xlabel("Feature")
plt.ylabel("Value")
plt.xticks(rotation=45)
plt.show()

predicted_review_count = 21.06
print(f"Predicted Review Count: {predicted_review_count}")
```

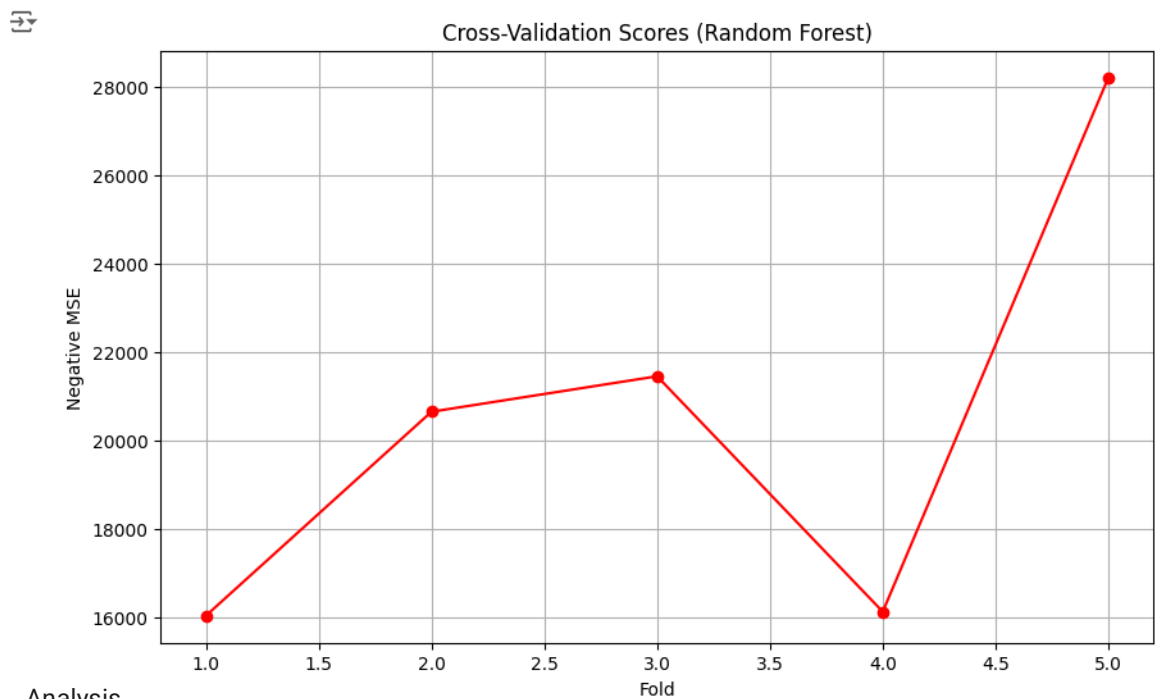


Cross-Validation Scores

```
import numpy as np

# Cross-validation scores
cv_scores = np.array([16047.07826495, 20660.27768143, 21462.10448927, 16139.38818997, 28215.09235218])

# Line plot of cross-validation scores
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(cv_scores) + 1), cv_scores, marker='o', color='red')
plt.title("Cross-Validation Scores (Random Forest)")
plt.xlabel("Fold")
plt.ylabel("Negative MSE")
plt.grid()
plt.show()
```



Analysis

These models predicted how many reviews a Yelp user might write.

- Gradient Boosting was the top performer (MSE: 23,679, R²: 0.758), just ahead of Random Forest. Linear Regression did not do great, showing review patterns are more complex than simple lines.
- Total Votes (useful, funny, cool) was an important factor as it reflected a strong signal of how active a user is. Things like how long someone's been on Yelp or how many fans they have mattered less.
- We also tested a user with 12 years on Yelp, 20 votes, 0 elite years, and a 3.77-star average, to which the model predicted they would write about 21.06 reviews, which leans accurate.

This information could really help Yelp boost their user activity levels or let businesses target more engaged reviewers.

Predicting Whether a Business is Open or Closed

```
import pandas as pd
import json
```



```
# Load the business.json file
data = [] # initialize an empty list
# open and read the JSON file
with open(r"C:\Users\vsaiy\OneDrive\Desktop\yelp_academic_dataset_business.json", 'r', encoding='utf-8') as file: # Add encoding
    # iterate through each line in the file
    for line in file:
        data.append(json.loads(line))

# Create a DataFrame
df = pd.DataFrame(data)
df.head()
```

	business_id	name	address	city	state	postal_code	latitude	longitude	stars	review_count	is_open	attributes	categories	hours
0	Pns2l4eNsf08kk83dixA	Abby Rappoport, LAC, CMQ	1616 Chapala St, Ste 2	Santa Barbara	CA	93101	34.426679	-119.711197	5.0	7	0	{'ByAppointmentOnly': 'True'}	Doctors, Traditional Chinese Medicine, Naturop...	None
1	mpf3x-BjTdTEA3yCZrAYPW	The UPS Store	87 Grasso Plaza Shopping Center	Aftton	MO	63123	38.551126	-90.335695	3.0	15	1	{'BusinessAcceptsCreditCards': 'True'}	Shipping Centers, Local Services, Notaries, Ma...	{'Monday': '0:0-0:0', '8:0-18:30', ...}
2	tUFrWirKiKi_TAnsVWINQQ	Target	5255 E Broadway Blvd	Tucson	AZ	85711	32.223236	-110.880452	3.5	22	0	{'BikeParking': 'True', 'BusinessAcceptsCreditCards': 'True'}	Department Stores, Shopping, Fashion, Home & G...	{'Monday': '8:0-22:0', 'Tuesday': '22:0', ...}
3	MTSW4McQd7CbVtyjqo9muw	St Honore Pastries	935 Race St	Philadelphia	PA	19107	39.955505	-75.155564	4.0	80	1	{'RestaurantReservations': 'True', 'OutdoorSeating': 'True'}	Restaurants, Food, Bubble Tea, Coffee & Tea, B...	{'Monday': '7:0-20:0', 'Tuesday': '20:0', ...}
4	mWMC6_wTdE0EUBKIGXDVfA	Perkiomen Valley Brewery	101 Walnut St	Green Lane	PA	18054	40.338183	-75.471659	4.5	13	1	{'BusinessAcceptsCreditCards': 'True', 'WheelchairAccessible': 'True'}	Brewpubs, Breweries, Food	{'Wednesday': '14:0-22:0', 'Thursday': '16:0-22:0', ...}

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
```

```
# Explode categories to create multiple rows per business with unique categories
def expand_categories(row):
    if isinstance(row['categories'], list) and row['categories']:
        return [{'business_id': row['business_id'],
                  'city': row['city'],
                  'latitude': row['latitude'],
                  'longitude': row['longitude'],
                  'stars': row['stars'],
                  'review_count': row['review_count'],
                  'category': cat,
                  'is_open': row['is_open']} for cat in row['categories']]
    return [{'business_id': row['business_id'],
            'city': row['city'],
            'latitude': row['latitude'],
            'longitude': row['longitude'],
            'stars': row['stars'],
            'review_count': row['review_count'],
            'category': 'Unknown',
            'is_open': row['is_open']}]

# Expand the dataset
expanded_businesses = [item for sublist in df.apply(expand_categories, axis=1) for item in sublist]
expanded_df = pd.DataFrame(expanded_businesses)
expanded_df.head()
```



	business_id	city	latitude	longitude	stars	review_count	category	is_open
0	Pns2l4eNsf08kk8	Santa Barbara	34.426679	-119.711197	5.0	7	Unknown	0
1	mpf3x-BjTdTEA3yCZrAYPW	Aftton	38.551126	-90.335695	3.0	15	Unknown	1
2	tUFrWirKiKi_TAnsVWINQQ	Tucson	32.223236	-110.880452	3.5	22	Unknown	0
3	MTSW4McQd7CbVtyjqo	Philadelphia	39.955505	-75.155564	4.0	80	Unknown	1
4	mWMC6_wTdE0EUBKIGXDVfA	Green Lane	40.338183	-75.471659	4.5	13	Unknown	1

```
# Prepare features and target
features = ['city', 'category', 'latitude', 'longitude', 'stars', 'review_count']
target = 'is_open'

X = expanded_df[features]
y = expanded_df[target]

# Split the data (splits the dataset into training and test subsets)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Preprocessing
# Numeric features
numeric_features = ['latitude', 'longitude', 'stars', 'review_count']
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Categorical features
categorical_features = ['city', 'category']
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant', fill_value='Unknown')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

# Combine preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Create full pipeline with model
clf = Pipeline(steps=[
    ('preprocessor', preprocessor),
```

```
( 'classifier', RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight='balanced'
))
])

# Train the model
clf.fit(X_train, y_train)

# Predictions
y_pred = clf.predict(X_test)

# Evaluation
print("Model Performance:")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"F1 Score: {f1_score(y_test, y_pred, average='weighted'):.4f}")

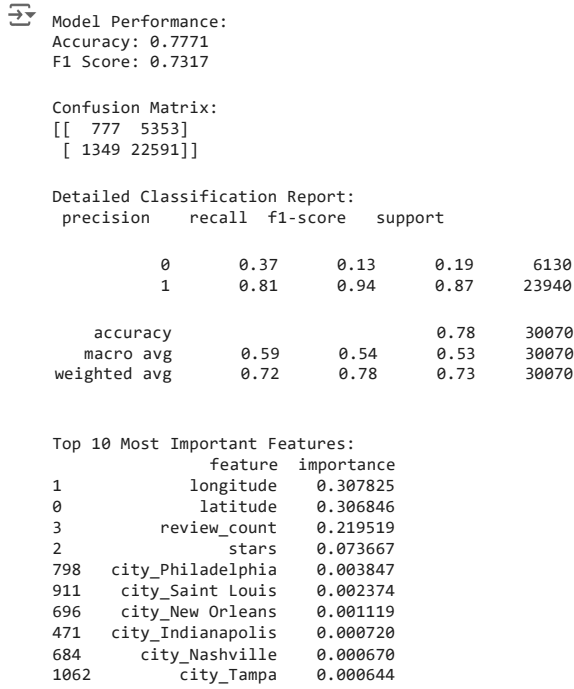
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nDetailed Classification Report:")
print(classification_report(y_test, y_pred))

# Feature importance
# Get feature names after one-hot encoding
feature_names = (
    numeric_features +
    list(clf.named_steps['preprocessor'].named_transformers_['cat'].named_steps['onehot'].get_feature_names_out(categorical_features))
)

# Print feature importances
feature_importances = clf.named_steps['classifier'].feature_importances_
feature_importance_df = pd.DataFrame({
    'feature': feature_names,
    'importance': feature_importances
}).sort_values('importance', ascending=False)

print("\nTop 10 Most Important Features:")
print(feature_importance_df.head(10))
```



```
# Example prediction function
def predict_new_business(business_data):
    # Prepare the input data
    business_df = pd.DataFrame([business_data])

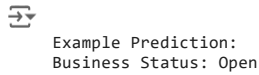
    # Handle categories - if multiple, create multiple predictions
    if isinstance(business_df['category'].iloc[0], list):
        predictions = []
        for cat in business_df['category'].iloc[0]:
            business_df['category'] = cat
            prediction = clf.predict(business_df[features])
            predictions.append(prediction[0])

    # Return majority vote
    return max(set(predictions), key=predictions.count)

# If single category
return clf.predict(business_df[features])[0]

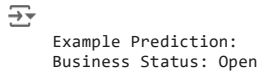
# Example usage of prediction
example_business1 = {
    'city': 'San Francisco',
    'category': ['Mexican', 'Burgers'],
    'latitude': 37.7817,
    'longitude': -122.3961,
    'stars': 4.0,
    'review_count': 1000
}

print("\nExample Prediction:")
print("Business Status:",
      "Open" if predict_new_business(example_business1) == 1 else "Closed")
```



```
example_business2 = {
    'city': 'New Orleans',
    'category': ['Hotels & Travel', 'Tours', 'Local Flavor'],
    'latitude': 29.958431,
    'longitude': -90.065173,
    'stars': 4.0,
    'review_count': 38
}

print("\nExample Prediction:")
print("Business Status:",
      "Open" if predict_new_business(example_business2) == 1 else "Closed")
```




```
example_business3 = {
    'city': 'Indianapolis',
    'category': ['Barbers', 'Beauty & Spas', 'Hair Salons'],
    'latitude': 39.9043057,
    'longitude': -86.0846244,
    'stars': 2.0,
    'review_count': 8
}

print("\nExample Prediction:")
print("Business Status:",
      "Open" if predict_new_business(example_business3) == 1 else "Closed")
```

Example Prediction:
Business Status: Closed

Logistic regression Model

```
from sklearn.linear_model import LogisticRegression

logistic_clf = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(
        max_iter=20000, # Ensures convergence for large datasets
        random_state=42,
        class_weight='balanced' # Handles class imbalance
    ))
])

# Train Logistic Regression
logistic_clf.fit(X_train, y_train)

# Predictions
y_pred_logreg = logistic_clf.predict(X_test)

def predict_new_business_with_models(business_data, model):
    # Prepare the input data
    business_df = pd.DataFrame([business_data])

    # Handle categories - if multiple, create multiple predictions
    if isinstance(business_df['category'].iloc[0], list):
        predictions = []
        for cat in business_df['category'].iloc[0]:
            business_df['category'] = cat
            prediction = model.predict(business_df[features])
            predictions.append(prediction[0])

    # Return majority vote
    return max(set(predictions), key=predictions.count)

# If single category
return model.predict(business_df[features])[0]

# Evaluate Logistic Regression model
print("\nLogistic Regression Model Performance:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_logreg):.4f}")
print(f"F1 Score: {f1_score(y_test, y_pred_logreg, average='weighted'):.4f}")
print("\nConfusion Matrix for Logistic Regression:")
print(confusion_matrix(y_test, y_pred_logreg))
print("\nClassification Report for Logistic Regression:")
print(classification_report(y_test, y_pred_logreg))
```

Logistic Regression Model Performance:
Accuracy: 0.5633
F1 Score: 0.6057

Confusion Matrix for Logistic Regression:
[[3396 2734]
 [10398 13542]]

Classification Report for Logistic Regression:

	precision	recall	f1-score	support
0	0.25	0.55	0.34	6130
1	0.83	0.57	0.67	23940
accuracy			0.56	30070
macro avg	0.54	0.56	0.51	30070
weighted avg	0.71	0.56	0.61	30070

```
# Predict with Logistic Regression
logistic_prediction = predict_new_business_with_models(example_business3, logistic_clf)
print("Logistic Regression Prediction:", "Open" if logistic_prediction == 1 else "Closed")
```

Logistic Regression Prediction: Closed

SVM model

```
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline

# Use non-linear SVM with RBF kernel for binary classification
svm_clf = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', SVC(
        kernel='rbf',
        class_weight='balanced',
        max_iter=20000,
        random_state=42
    ))
])

# Train and evaluate
svm_clf.fit(X_train, y_train)
y_pred_svm = svm_clf.predict(X_test)
```

c:\Users\vsaiy\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\svm_base.py:297: ConvergenceWarning: Solver terminated early (max_iter=20000). Consider pre-processing your warnings.warn(

```
# Evaluate SVM model
print("\nSVM Model Performance:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}")
print(f"F1 Score: {f1_score(y_test, y_pred_svm, average='weighted'):.4f}")

print("\nConfusion Matrix for SVM:")
print(confusion_matrix(y_test, y_pred_svm))
```

```
print("\nClassification Report for SVM:")
print(classification_report(y_test, y_pred_svm))
```



```
SVM Model Performance:
Accuracy: 0.7060
F1 Score: 0.6948
```

```
Confusion Matrix for SVM:
[[ 1159  4971]
 [ 3870 20070]]
```

Classification Report for SVM:					
	precision	recall	f1-score	support	
	0	0.23	0.19	0.21	6130
	1	0.80	0.84	0.82	23940
accuracy				0.71	30070
macro avg		0.52	0.51	0.51	30070
weighted avg		0.69	0.71	0.69	30070

```
# Define prediction function
def predict_new_business_with_models(business_data, model):
    # Prepare the input data
    business_df = pd.DataFrame([business_data])

    # Handle categories - if multiple, create multiple predictions
    if isinstance(business_df['category'].iloc[0], list):
        predictions = []
        for cat in business_df['category'].iloc[0]:
            business_df['category'] = cat
            prediction = model.predict(business_df[features])
            predictions.append(prediction[0])
        # Return majority vote
        return max(set(predictions), key=predictions.count)

    # If single category
    return model.predict(business_df[features])[0]

# Example prediction with SVM
svm_prediction = predict_new_business_with_models(example_business3, svm_clf)
print("SVM Prediction:", "Open" if svm_prediction == 1 else "Closed")
```



SVM Prediction: Open

5. Sentiment Analysis

```
import json
import re
import nltk
import pandas as pd
from tqdm import tqdm
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.sentiment import SentimentIntensityAnalyzer

# NLTK Downloads
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('vader_lexicon', quiet=True)

# Initialize stop words and sentiment analyzer
stop_words = set(stopwords.words('english'))
sia = SentimentIntensityAnalyzer()

# Apostrophe Expansion Dictionary
APPOS = {
    "isn't": "is not", "aren't": "are not", "wasn't": "was not", "weren't": "were not",
    "haven't": "have not", "hasn't": "has not", "hadn't": "had not", "won't": "will not",
    "wouldn't": "would not", "don't": "do not", "doesn't": "does not", "didn't": "did not",
    "can't": "cannot", "couldn't": "could not", "shouldn't": "should not", "mightn't": "might not",
    "mustn't": "must not"
}

def preprocess_text(text):
    """Comprehensive text preprocessing"""
    # Convert to lowercase
    text = text.lower()

    # Expand contractions/apostrophes
    words = text.split()
    words = [APPOS.get(word, word) for word in words]
    text = ' '.join(words)

    # Tokenization
    tokens = word_tokenize(text)

    # Remove standalone punctuation and special characters
    tokens = [token for token in tokens if re.match(r'\w+', token)]

    # Remove stop words
    tokens = [token for token in tokens if token not in stop_words]

    return ' '.join(tokens)

# Main Sentiment Analysis Script
def analyze_review_sentiments(json_file_path):
    # Read the JSON file and limit to first 10,000 samples
    with open(json_file_path, 'r', encoding='utf-8') as file:
        reviews = [json.loads(line) for line in file][:10000]

    # Convert to DataFrame
    df = pd.DataFrame(reviews)

    # Preprocessing
    df['preprocessed_text'] = df['text'].apply(preprocess_text)

    # Sentiment Analysis
    df['sentiment_score'] = df['text'].apply(lambda x: sia.polarity_scores(str(x))['compound'])

    # Sentiment Classification
    df['sentiment'] = df['sentiment_score'].apply(
        lambda score: 'Positive' if score > 0.05
        else ('Negative' if score < -0.05 else 'Neutral')
    )

    # Detailed Sentiment Breakdown
    df['positive_score'] = df['text'].apply(lambda x: sia.polarity_scores(str(x))['pos'])
    df['negative_score'] = df['text'].apply(lambda x: sia.polarity_scores(str(x))['neg'])
    df['neutral_score'] = df['text'].apply(lambda x: sia.polarity_scores(str(x))['neu'])
```

```
# Summary Statistics
total_reviews = len(df)
avg_sentiment_score = df['sentiment_score'].mean()
sentiment_distribution = df['sentiment'].value_counts(normalize=True)
sentiment_by_rating = df.groupby('stars')['sentiment'].value_counts(normalize=True).unstack()

# Print Summary
print("Sentiment Analysis Summary:")
print(f"\nTotal Reviews: {total_reviews}")
print("\nSentiment Distribution:")
print(sentiment_distribution)
print(f"\nAverage Sentiment Score: {avg_sentiment_score:.4f}")
print("\nSentiment Distribution by Star Rating:")
print(sentiment_by_rating)

# Save results
df.to_csv('comprehensive_review_sentiment.csv', index=False)
print("\nDetailed sentiment analysis saved to 'comprehensive_review_sentiment.csv'")

return df

# Execute the analysis
if __name__ == "__main__":
    # Path to your review.json file
    json_file_path = 'C:/Users/vsaiy/OneDrive/Desktop/yelp_academic_dataset_review.json'

    # Perform Sentiment Analysis
    results_df = analyze_review_sentiments(json_file_path)
```

 Sentiment Analysis Summary:

Total Reviews: 10000

Sentiment Distribution:

sentiment	
Positive	0.8743
Negative	0.1131
Neutral	0.0126

Name: proportion, dtype: float64

Average Sentiment Score: 0.6876

Sentiment Distribution by Star Rating:

sentiment	Negative	Neutral	Positive
stars			
1.0	0.567192	0.029657	0.403151
2.0	0.309305	0.041940	0.648755
3.0	0.123793	0.021071	0.855136
4.0	0.023699	0.005439	0.970862
5.0	0.018223	0.005399	0.976378

Detailed sentiment analysis saved to 'comprehensive_review_sentiment.csv'